# Aprendizaje por refuerzo

Tercera gran familia: un agente aprende **probando**, con recompensas y castigos. Lo vemos con una demostración guiada en el hotel.

# El bucle del refuerzo

### Agente

Quien decide (aquí, un empleado que quiere llegar a una habitación).

### Entorno

El pasillo del hotel: estados (casillas) y acciones (arriba, abajo, izquierda, derecha).

### Recompensa

+ si llega a la habitación con vistas; − por zona en obras (−5) y por cada paso (coste).

No hay respuestas correctas dadas: el agente aprende **qué acciones llevan a más recompensa total**.

# El pasillo del hotel (cuaderno S09_04)

```
S  .  .  .  .      S = recepción (inicio)
.  #  #  .  X      # = pared
.  .  .  .  .      X = zona en obras (−5, se puede pisar)
.  #  .  .  G      G = habitación con vistas al mar (meta)
```

Cuadrícula de 4 × 5 = **20 estados**. La clase `EntornoHotel` y la función `entrenar` ya están escritas; tú programas `actualizar_q` y `elegir_accion`.

# La tabla Q y su regla de actualización

`Q[estado, acción]` = recompensa total esperada si estoy aquí, hago esta acción y luego sigo lo mejor posible. Empieza en ceros.

```
def actualizar_q(Q, s, a, r, s2, alpha, gamma):              # Ejercicio 1
    objetivo = r + gamma * Q[s2].max()      # lo que ahora crees que vale la acción
    Q[s, a] += alpha * (objetivo - Q[s, a])  # mueve Q una fracción alpha hacia ese valor
```

- `alpha` (0–1): cuánto corrige cada experiencia.
- `gamma` (0–1): cuánto pesa el futuro frente al presente.

# Explorar o explotar: epsilon-greedy

```
def elegir_accion(Q, s, epsilon, rng):                    # Ejercicio 2
    if rng.random() < epsilon:
        return int(rng.integers(4))                       # explorar: al azar
    return int(np.argmax(Q[s]))                           # explotar: la mejor
```

Si siempre explota, **nunca descubre** caminos mejores. Si siempre experimenta, **nunca aprovecha** lo aprendido.

# Política aprendida y experimento

```
politica = Q.argmax(axis=1)          # Ejercicio 3: mejor acción en cada estado (20 acciones)
```

- `camino`: estados que visita el agente **sin explorar** desde recepción hasta la meta.
- **Experimento**: con `coste_paso=0` la única recompensa llega al final; sin exploración el agente **no aprende**. La exploración importa.
- Mini-reto: pasar por la zona en obras cuesta **−20** (reclamaciones). Copia `EntornoHotel`, reentrena con `clase_entorno` y comprueba si el camino cambia.

# Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

## S09_04_q_learning

30 min

Q-learning en el hotel (demostración guiada)

sprints/S09/notebooks/S09_04_q_learning.ipynb

RETO

## Reto del sprint

40 min

Recomendador, ficha de segmentos, IA responsable y ensayo (pasos 4–5)

sprints/S09/guia/Reto_Sprint_09.md